

Programmation fonctionnelle 2
Preuves et programmes
CM10 : `bool` *versus* `Prop`

Pierre Rousselin

Université Sorbonne Paris Nord

avril 2026

Prédicats inductifs (rappels)

Inférieur ou égal dans `nat`

`leb` à valeurs dans `bool`

Réflexion entre `le` et `leb`

Le fameux prédicat E de M. Lenglet

```
Inductive E : Z -> Z -> Prop :=  
| ZZ : E 0 0  
| UN (x y : Z) (h : E x y) : E (x - 1) (y + 1)  
| DEUX (x y : Z) (h : E x y) : E (x + 2) (y - 2).
```

ou, avec une autre notation équivalente :

```
Inductive E : Z -> Z -> Prop :=  
| ZZ : E 0 0  
| UN (x y : Z) : E x y -> E (x - 1) (y + 1)  
| DEUX (x y : Z) : E x y -> E (x + 2) (y - 2).
```

Prédicat inductif et règles d'inférence

- ▶ On a défini une relation binaire sur $\mathbb{Z}$
- ▶ avec un cas de base :

$$\frac{}{\Gamma \vdash E \ 0 \ 0} \text{ (ZZ)}$$

- ▶ et deux autres règles d'inférence :

$$\frac{\Gamma \vdash E \ x \ y}{\Gamma \vdash E \ (x - 1) \ (y + 1)} \text{ (UN)}$$

$$\frac{\Gamma \vdash E \ x \ y}{\Gamma \vdash E \ (x + 2) \ (y - 2)} \text{ (DEUX)}$$

Utiliser les règles pour prouver une relation

- On peut utiliser ces règles comme n'importe quel constructeur pour prouver une relation du type $E\ x\ y$.

Lemma Eex1 : $E\ 3\ (-3)$.

Proof.

`apply (DEUX 1 (-1)).`

`apply (DEUX (-1) 1).`

`apply (UN 0 0).`

`exact ZZ.`

Qed.

- **Lemma** Eex2 $(x\ y : Z) : E\ x\ y \rightarrow E\ (x + 1)\ (y - 1)$.

Proof.

`intros H.`

`replace (x + 1) with ((x + 2) - 1) by lia.`

`replace (y - 1) with ((y - 2) + 1) by lia.`

`apply UN.`

`apply DEUX.`

`exact H.`

Qed.

Minimalité de l'inductif

- ▶ Mais les règles seules ne disent pas comment raisonner sur x et y satisfaisant $E\ x\ y$.
- ▶ Le fait qu'on puisse `match` sur les constructeurs et définir des fonctions récursives entraîne la **minimalité** d'un tel prédicat. Notre prédicat est le plus petit satisfaisant les trois règles précédentes.
- ▶ Lorsqu'on définit un type **Inductive**, Rocq génère automatiquement un principe d'induction (construit avec `match` et, si besoin, une fonction récursive) qui prouvent cette minimalité.
- ▶ Il a toujours le nom `type_ind`, où `type` est le type défini inductivement.

E_ind

Pour notre ensemble E, le type de E_ind est

E_ind

```
: forall P : Z -> Z -> Prop,  
  P 0 0 ->  
  (forall x y : Z, E x y -> P x y -> P (x - 1) (y + 1)) ->  
  (forall x y : Z, E x y -> P x y -> P (x + 2) (y - 2)) ->  
  forall z z0 : Z, E z z0 -> P z z0
```

- ▶ correspond à cette minimalité
- ▶ utilisé (par défaut) par les tactiques `induction` et `destruct` lorsqu'on raisonne sur un terme (une preuve) H de type E x y.

Prédicats inductifs (rappels)

Inférieur ou égal dans **nat**

leb à valeurs dans **bool**

Réflexion entre **le** et **leb**

Peano.le

- ▶ Il y a sans doute 100 façons de définir le prédicat (binaire) $\leq$ sur les entiers de Peano.
- ▶ Ce qui a été choisi historiquement, chargé dans le prélude, est :

```
Inductive le (n : nat) : nat -> Prop :=  
  | le_n : le n n  
  | le_S (m : nat) : le n m -> le n S m.  
Notation "n <= m" := (le n m).
```

- ▶ qui correspond aux règles d'inférence :

$$\frac{}{\Gamma \vdash n \leq n} (\text{le_n})$$
$$\frac{n \leq m}{\Gamma \vdash n \leq S\ m} (\text{le_S})$$

Exemples de preuves

► `Lemma le_exple1 : 2 <= 5.`

Exemples de preuves

- ▶ `Lemma le_exple1 : 2 <= 5.`
- ▶ `Proof.`
 - `apply le_S. (* 2 <= 4 *)`
 - `apply le_S. (* 2 <= 3 *)`
 - `apply le_S.`
 - `exact (le_n 2).`
- ▶ `Qed.`
- ▶ `Lemma le_refl (n : nat) : n <= n.`

Exemples de preuves

- ▶ `Lemma le_exple1 : 2 <= 5.`
- ▶ `Proof.`
 - `apply le_S. (* 2 <= 4 *)`
 - `apply le_S. (* 2 <= 3 *)`
 - `apply le_S.`
 - `exact (le_n 2).`
- `Qed.`
- ▶ `Lemma le_refl (n : nat) : n <= n.`
- ▶ `Proof. exact (le_n n). Qed.`

Exemples avec induction

- Récurrence sur n :

Lemma `le_0_1 (n : nat) : 0 <= n.`

Exemples avec induction

- Récurrence sur n :

```
Lemma le_0_1 (n : nat) : 0 <= n.
```

- Proof.

```
  induction n as [| p IH].  
  - exact (le_n 0).  
  - apply le_S. exact IH.  
Qed.
```

- Induction sur le :

```
Lemma le_le_succ_succ (n m : nat) : n <= m -> S n <= S m.
```

Exemples avec induction

- Récurrence sur n :

```
Lemma le_0_1 (n : nat) : 0 <= n.
```

- Proof.

```
  induction n as [| p IH].  
  - exact (le_n 0).  
  - apply le_S. exact IH.  
Qed.
```

- Induction sur le :

```
Lemma le_le_succ_succ (n m : nat) : n <= m -> S n <= S m.
```

- Proof.

```
  intros H.  
  induction H as [| m H IH].  
  - exact (le_n _).  
  - apply le_S. exact IH.  
Qed.
```

Prédicats inductifs (rappels)

Inférieur ou égal dans `nat`

`leb` à valeurs dans `bool`

Réflexion entre `le` et `leb`

Motivation

La définition inductive de **le** dans **Prop** est simple et assez élégante. Mais elle présente aussi quelques inconvénients :

- ▶ Ne passe pas par le calcul : on prouve « à la main » que $2 \leq 5$.
- ▶ On ne peut donc pas définir de « vraies » fonctions avec par exemple, un cas où $n \leq m$ et un autre où ce n'est pas le cas.
- ▶ Par ailleurs, sans tiers exclu par défaut, on n'a pas gratuitement que $(n \leq m) \vee \neg(n \leq m)$.
- ▶ Il est pourtant clair qu'on peut toujours répondre à la question « est-ce que $n \leq m$? » lorsque n et m sont deux entiers concrets.

Fonction `leb`

- ▶ On définit donc une fonction `leb : nat -> nat -> bool`.

Fonction leb

- ▶ On définit donc une fonction `leb : nat -> nat -> bool`.
- ▶ **Fixpoint** `leb (n m : nat) :=`
 `match n with`
 | `0 => true`
 | `S p => match m with`
 | `0 => false`
 | `S q => leb p q`
 `end`

 `end.`

 Notation `"x <=? y" := (leb x y).`

Fonction leb

- ▶ On définit donc une fonction `leb : nat -> nat -> bool`.
- ▶ **Fixpoint** `leb (n m : nat) :=`
 `match n with`
 | `0 => true`
 | `S p => match m with`
 | `0 => false`
 | `S q => leb p q`
 `end`

 `end.`
 Notation `"x <=? y" := (leb x y).`
- ▶ Mais là on n'a plus aucune propriété gratuitement
- ▶ et pour l'instant, aucun rapport entre $\leq$ et `<=?`.

Preuves sur leb

► **Lemma** `leb_refl (n : nat) : n <=? n = true.`

Preuves sur leb

- ▶ `Lemma leb_refl (n : nat) : n <=? n = true.`
- ▶ `Proof.`
 - `induction n as [| p IH].`
 - `- reflexivity.`
 - `- simpl; rewrite IH; reflexivity.`
- `Qed.`
- ▶ `Lemma leb_leb_succ_r (n m : nat) :`
 - `n <=? m = true -> n <=? S m = true.`

Preuves sur leb

► `Lemma leb_refl (n : nat) : n <=? n = true.`

► `Proof.`

`induction n as [| p IH].`

`- reflexivity.`

`- simpl; rewrite IH; reflexivity.`

`Qed.`

► `Lemma leb_leb_succ_r (n m : nat) :`
`n <=? m = true -> n <=? S m = true.`

► `Proof.`

`induction n as [| p IH] in m |- *.`

`- intros _; reflexivity.`

`- intros H. destruct m as [| m'].`

`+ discriminate H.`

`+ simpl. apply IH.`

`exact H.`

`Qed.`

Prédicats inductifs (rappels)

Inférieur ou égal dans `nat`

`leb` à valeurs dans `bool`

Réflexion entre `le` et `leb`

le et leb

► **Lemma** `le_leb (n m : nat) : n <= m -> n <=? m = true.`

le et leb

- ▶ `Lemma le_leb (n m : nat) : n <= m -> n <=? m = true.`
- ▶ `Proof.`
 - `intros H.`
 - `induction H as [| m H IHle].`
 - `- exact (leb_refl _).`
 - `- exact (leb_leb_succ_r n m IHle).`
- `Qed.`
- ▶ `Lemma leb_le (n m : nat) : n <=? m = true -> n <= m.`

le et leb

► **Lemma** `le_leb` $(n\ m : \text{nat}) : n \leq m \rightarrow n \leq? m = \text{true}$.

► **Proof.**

`intros H.`

`induction H as [| m H IHle].`

`- exact (leb_refl _).`

`- exact (leb_leb_succ_r n m IHle).`

`Qed.`

► **Lemma** `leb_le` $(n\ m : \text{nat}) : n \leq? m = \text{true} \rightarrow n \leq m$.

► **Proof.**

`induction n as [| p IH] in m |- *.`

`- intros _. exact (le_0_1 m).`

`- intros H. destruct m as [| m'].`

`* discriminate H.`

`* apply le_le_succ_succ, IH.`

`exact H.`

`Qed.`

Applications

- Preuve par calcul :

```
Lemma exple_le_leb : 100 <= 200.
```

```
Proof.
```

```
  apply leb_le.
```

```
  simpl.
```

```
  reflexivity.
```

```
Qed.
```

- Décidabilité (tiers exclu) pour le :

```
Lemma le_decidable (n m : nat) : n <= m \/ (~ n <= m).
```

```
Proof.
```

```
  destruct (n <=? m) eqn:E.
```

```
  - left.
```

```
    apply leb_le. exact E.
```

```
  - right. intros H. apply le_leb in H.
```

```
    rewrite E in H. discriminate H.
```

```
Qed.
```

Et au-delà

- ▶ Fonction qui trie une liste de `nat` : on a besoin de calculer pour savoir si un entier est inférieur ou égal à un autre : on utilise `leb`.
- ▶ Preuve que la liste finale est bien triée : c'est une propriété, on utilise `le`.
- ▶ Lien entre les deux : la réflexion, ici `le_leb` et `leb_le`.